

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Título del Proyecto:

Sistema Inteligente de Seguridad y Gestión de Tráfico Urbano: Cimentación y Monitoreo Local FASE2

PONDERACIÓN: 35

Horas Aproximadas: 29

Índice

1. Resumen Ejecutivo	3
2. Competencia que desarrollaremos	3
3. Objetivos del Aprendizaje	4
3.1 Objetivo General.....	4
3.2 Objetivos Específicos	4
4. Enunciado del Proyecto	5
4.1 Descripción del problema a resolver.....	5
4.2 Alcance del proyecto	5
4.3 Entregables	10
5. Metodología	10
6. Desarrollo de Habilidades Blandas.....	11
6.1 Proyectos en Grupo.....	12
7. Cronograma	12
8. Rúbrica de Calificación	13
8.1 Requisitos para optar a la calificación.....	13
8.2 Valores	14
8.3 Comentarios Generales.....	14

1. Resumen Ejecutivo

Este proyecto aborda los desafíos de gestión del tráfico urbano y seguridad ciudadana en zonas históricas de alta densidad, como el Centro Histórico de la Zona 1 de Guatemala, donde la movilidad ineficiente y la falta de vigilancia proactiva generan riesgos para la población y el patrimonio.

El sistema propuesto es una solución integrada basada en Internet de las Cosas (IoT) e Inteligencia Artificial (IA) que permite monitorear en tiempo real el flujo vehicular, detectar incendios, activar alertas de emergencia y reconocer comportamientos de riesgo, como vehículos que cruzan semáforos en rojo o personas en listas de vigilancia.

La solución se compone de sensores conectados a microcontroladores Arduino, que capturan datos ambientales y de tráfico. Estos se comunican mediante conexión serial con una Raspberry Pi, que actúa como nodo central de comunicación y publica la información vía MQTT. Las tareas de procesamiento de video (reconocimiento de placas y rostros) son realizadas por computadoras externas que ejecutan modelos de IA, enviando los resultados a la Raspberry Pi para su integración y almacenamiento.

Los datos son gestionados por una API y una base de datos, y son accesibles para los ciudadanos a través de una aplicación móvil desarrollada en Flutter, que muestra alertas, estado del tráfico y tiempo de llegada del transporte público. Además, se implementa un sistema de códigos QR en puntos históricos para fomentar el turismo mediante información digital.

En resumen, este proyecto desarrolla un sistema inteligente, escalable y conectado que mejora la seguridad, optimiza la movilidad y promueve la cultura, utilizando tecnología accesible y arquitecturas modernas de IoT y computación distribuida.

2. Competencia que desarrollaremos

- Integra soluciones IoT utilizando hardware, software y análisis de datos mediante metodologías activas como ABP (aprendizaje basado en proyectos) para resolver problemas prácticos en áreas como automatización y monitoreo ambiental.
- Configura un sistema IoT basado en Arduino y Raspberry Pi utilizando protocolos como MQTT y bibliotecas específicas para implementar comunicación segura entre dispositivos embebidos.
- Diseña una interfaz gráfica para visualización de datos en tiempo real usando herramientas como Processing o p5.js para representar información recolectada desde sensores conectados a Arduino.
- Implementa rutinas de interrupción en sistemas embebidos usando Arduino IDE y bibliotecas específicas para gestionar eventos asíncronos de sensores y actuadores en aplicaciones de automatización que requieren respuesta en tiempo real.
- Configura sistemas de comunicación IoT utilizando protocolos como HTTP o COAP mediante servidores API RESTful y dispositivos embebidos para optimizar la transmisión de datos en redes inalámbricas de bajo consumo.

3. Objetivos del Aprendizaje

3.1 Objetivo General

Diseñar e implementar un sistema inteligente de seguridad y gestión de tráfico urbano basado en IoT e Inteligencia Artificial, que integre sensores, microcontroladores y computadoras externas para monitorear condiciones ambientales, detectar infracciones y emergencias, y proporcionar información en tiempo real a través de una aplicación , utilizando una Raspberry Pi como nodo central de comunicación y coordinación, sin ejecutar modelos de IA directamente sobre ella.

3.2 Objetivos Específicos

- Diseñar y construir un prototipo a escala de un sistema de seguridad y gestión de tráfico, utilizando microcontroladores Arduino para el control local y la captura de datos de sensores.
- Implementar un framework de comunicación IoT basado en el protocolo MQTT para garantizar la transmisión de datos de manera eficiente y desacoplada entre los dispositivos de campo, el servidor y las aplicaciones cliente.
- Desarrollar una plataforma de software centralizada, que incluya una base de datos (SQL o NoSQL) para el almacenamiento histórico de eventos y una API para el consumo de datos.
- Crear una aplicación móvil multiplataforma (Flutter) que sirva como interfaz para el ciudadano, mostrando información en tiempo real del estado del tráfico, la ubicación simulada del transporte público y alertas de seguridad.
- Integrar un módulo de Inteligencia Artificial en computadoras externas, que procesarán video en tiempo real y enviarán los resultados a la Raspberry Pi, la cual actuará como nodo central de integración y comunicación.
- Fomentar el turismo y la cultura mediante la implementación de un sistema de códigos QR en puntos de interés histórico, vinculados a una página web informativa.

4. Enunciado del Proyecto

4.1 Descripción del problema a resolver

El presente proyecto propone el diseño, desarrollo e implementación de un sistema integral para la gestión inteligente del tráfico y la seguridad ciudadana en un entorno urbano simulado, basado en el Centro Histórico de la Zona 1 de Guatemala. Ante los crecientes desafíos en materia de movilidad y seguridad en áreas de alta densidad poblacional y valor patrimonial, se hace fundamental explorar soluciones tecnológicas innovadoras.

Este sistema se fundamenta en la tecnología de Internet de las Cosas (IoT) para la recopilación de datos en tiempo real a través de una red de sensores distribuidos. La información recolectada abarca desde el flujo vehicular, la detección de incendios, detección de sismo, hasta la activación de alertas de pánico. Posteriormente, los datos son procesados, almacenados y analizados para la toma de decisiones automatizada y la presentación de información relevante a los usuarios finales a través de una aplicación móvil.

La fase culminante del proyecto integra un componente de Inteligencia Artificial (IA) mediante el uso de una Raspberry Pi como nodo central de comunicación, mientras que las tareas de procesamiento de video se realizarán en computadoras externas que enviarán la información procesada a la Raspberry Pi. Esto permite mantener un sistema escalable y eficiente, adaptándose mejor a las limitaciones de hardware de la plataforma embebida. El sistema permitirá funcionalidades avanzadas como el reconocimiento de matrículas para la detección de infracciones, el reconocimiento facial para la identificación de individuos en listas de vigilancia, armas blancas, fortaleciendo así la seguridad proactiva en el área de operación.

4.2 Alcance del proyecto

Esta tercera fase representa la culminación estratégica del proyecto, transformando la infraestructura IoT establecida en la Fase 2 en un sistema inteligente proactivo de seguridad urbana mediante la integración robusta de módulos de Inteligencia Artificial (IA). El objetivo principal es activar y orquestar capacidades avanzadas de detección visual (placas, rostros, armas) que enriquezcan la base de datos del sistema, generen alertas contextuales y ofrezcan nuevas capas de información analítica al ciudadano, todo ello sin comprometer la estabilidad del nodo central (Raspberry Pi), que actúa como pasarela y coordinador, delegando el procesamiento pesado de video a computadoras externas.

Componentes Tecnológicos

HARDWARE:

- **Raspberry Pi (Nodo Central):** Mantiene y amplía su rol como epicentro del sistema. En esta fase, se configura como un servidor API (192.168.0.5:3000/api/info) que recibe, valida y almacena los resultados de las detecciones enviadas por las computadoras externas. Su función es puramente de integración, comunicación y persistencia, sin ejecutar modelos de IA.
- **Computadoras Externas (Laptops):** Actúan como nodos de procesamiento de IA. Cada una ejecuta modelos de visión por computadora (YOLO, EasyOCR, MobileNet, etc.) de forma local. Pueden utilizar cámaras USB físicas o la aplicación DroidCam para aprovechar las cámaras de los teléfonos móviles como fuentes de video.

- **Microcontrolador Arduino UNO:** Mantiene su función de interfaz con el mundo físico. Se amplía su código para activar alertas sonoras diferenciadas en el zumbador local cuando recibe comandos específicos (vía serial desde la Raspberry Pi) relacionados con las nuevas detecciones de IA: "anti-robo" (persona en lista negra), "arma blanca" y "placas".
- **Sensores y Actuadores Existentes:** Todos los componentes de fases anteriores (sensores ultrasónicos(miden distancia), de gas, botón de pánico, piezoeléctrico, semáforos LED, zumbador) continúan operando y se integran con las nuevas funcionalidades.

SOFTWARE:

- **Backend (Desplegado en Raspberry Pi):**
 - **API RESTful (/api/info):** Se desarrolla o extiende una API (usando Flask, Express, etc.) que expone un endpoint POST para recibir los payloads de detección enviados por las laptops. Cada payload incluye el tipo de evento (placa, persona, arma), los metadatos relevantes (string de la placa, ID de la persona, tipo de arma) y la imagen. La API procesa esta información, la almacena en la base de datos y, si corresponde, envía un comando de alerta al Arduino.
 - **Base de Datos (SQL o NoSQL):** Se amplía el guardado de la información(obtenida por la IA):
 - **detecciones_placas_vehiculos:** Almacena el texto de la placa, la fecha/hora, la ubicación aproximada.
 - **detecciones_personas:** Almacena el ID de la persona (vinculado a una lista negra), la fecha/hora, la ubicación aproximada.
 - **detecciones_armas:** Almacena el tipo de arma blanca detectada, la fecha/hora, la ubicación aproximada.
 - **(Opcional) Servicio de Cola/Procesamiento:** Para manejar picos de carga, se puede implementar un servicio intermedio que reciba las solicitudes POST, las encole y las procese de forma asíncrona para no bloquear la API.
- **Módulo de Inteligencia Artificial (En Laptops Externas):**
 - **API Local de IA:** Cada laptop ejecuta un script o servicio ligero (por ejemplo, con Flask o FastAPI) que captura el flujo de video, aplica el modelo de IA correspondiente (uno para placas, otro para rostros, otro para armas) y, al detectar un objeto de interés, empaqueta la información y la envía vía POST a la API de la Raspberry Pi (192.168.0.5:3000/api/info).
 - **IA's a implementar:** se debe implementar 3 IA's las cuales sera:
 - **Deteccion de placas de vehiculos:** a la hora de pasarse un rojo del semáforo.
 - **Deteccion de rostros(una lista rostros):** un lista negra de personas / delincuentes.
 - **Deteccio de armas blancas:** al ver un civil con un arma blanca.
- **Frontend (Aplicación Móvil - Flutter):**
 - Se amplía la aplicación para incluir nuevas secciones y widgets:
 - **Nuevas Alertas en Tiempo Real:** La app ahora muestra mas notificaciones en tiempo real para los eventos de "Persona en Lista Negra Detectada" y "Arma Blanca Detectada", complementando las alertas existentes (semáforo, pánico, etc.).

- **Dashboard Analítico:** Se implementan nuevas vistas en el panel de control:
 - *Top de Personas (Lista Negra):* Muestra una lista o galería de las personas más frecuentemente detectadas en la zona, acompañadas de su imagen de rostro.
 - *Top de Armas Blancas:* Muestra un ranking o lista de los tipos de armas blancas más comúnmente detectados, con su imagen representativa o el nombre del tipo.
 - *Numero de infracciones:* muestra las infracciones y su placa durante el día al pasarse un rojo del semáforo.
- *La app sigue consumiendo datos históricos a través de la API REST para alimentar estas nuevas visualizaciones.*
- *Las demas funciones siguen manteniendose(mapa, graficas, etc..).*
- **Plataforma de Visualización (Grafana):**
 - *Se configura Grafana para conectarse a la base de datos. Se crean nuevos paneles y gráficos que visualizan en tiempo real (o con actualización periódica) de todas las métricas:*
- **Sistema de Turismo (QR y Web):**
 - *Se implementa el sistema de códigos QR en puntos históricos de la maqueta, vinculados a una página web informativa, cumpliendo con el objetivo de fomentar el turismo cultural.*

Descripción Detallada del Funcionamiento

El flujo de datos en esta fase integra la nueva capa de IA con la infraestructura existente, creando un ciclo de detección, alerta y análisis:

1. **Captura y Procesamiento de Video (Laptop Externa):**
Una laptop, usando una cámara (USB o DroidCam), captura video en tiempo real. Un script local ejecuta un modelo de IA (por ejemplo, YOLO para detectar armas). Al identificar un objeto de interés (ej: un cuchillo), el script toma una captura, extrae la información relevante (tipo: "cuchillo") y envía un POST a **192.168.0.5:3000/api/info** con un payload como: **{"tipo": "arma_blanca", "subtipo": "cuchillo", etc...}**.
2. **Integración y Persistencia (Raspberry Pi):**
La API en la Raspberry Pi recibe el POST. Valida el payload, guarda la información en la tabla **detecciones_armas** de la base de datos. Luego, para eventos críticos como "arma_blanca" o "persona_lista_negra", la API envía un comando específico (por ejemplo, **"ALERTA_ARMA"**) a través del puerto serial al Arduino, para enviar una alerta.
3. **Alerta Física Local (Arduino):**
El Arduino, en constante escucha del puerto serial, recibe el comando **"ALERTA_ARMA"**. Su código actualizado reconoce este comando y activa el zumbador con un patrón de sonido único y diferenciado para indicar que se ha detectado un arma blanca, proporcionando una retroalimentación auditiva inmediata en el entorno físico de la maqueta.
4. **Visualización y Análisis (Flutter y Grafana):**
 - *La aplicación Flutter, suscrita a las actualizaciones de la API (o consultando periódicamente), recibe la nueva alerta y la muestra al usuario en la sección de notificaciones.*
 - *Simultáneamente, Grafana, consultando la base de datos, actualiza sus gráficos. Por ejemplo, un gráfico de barras podría incrementar el conteo para "Cuchillos" en el periodo actual.*

- *En el dashboard de Flutter, por ejemplo la sección "Top de Armas Blancas" se actualiza para reflejar que "Cuchillo" es ahora el tipo más frecuente.*

5. **Consulta de Datos Históricos (Flutter):**

Cuando un usuario abre la sección "Top de Personas (Lista Negra)" en la app Flutter, esta realiza una llamada GET a un nuevo endpoint de la API (por ejemplo, [/api/top_personas](#)). La API consulta la base de datos, agrega los datos (IDs, imagen, etc..) y los devuelve a la app, que los renderiza en pantalla.

Este flujo demuestra cómo la Fase 3 cierra el ciclo, transformando datos visuales crudos en acciones físicas, alertas digitales y conocimiento analítico, consolidando un sistema de ciudad inteligente verdaderamente integral.

Aclaraciones

- **API**
 - **API CLOUD:** cloud que es la que transmite la información de.
 - MQTT->Base
 - Base->Flutter
 - **API Laptop:** son los que ejecutan los modelos de IA, envían a través de wifi el payload.
 - **API Raspberry:** es la que recibe la información del payload de las laptops y envía la información (solo de los modelos de IA) a la base de datos.

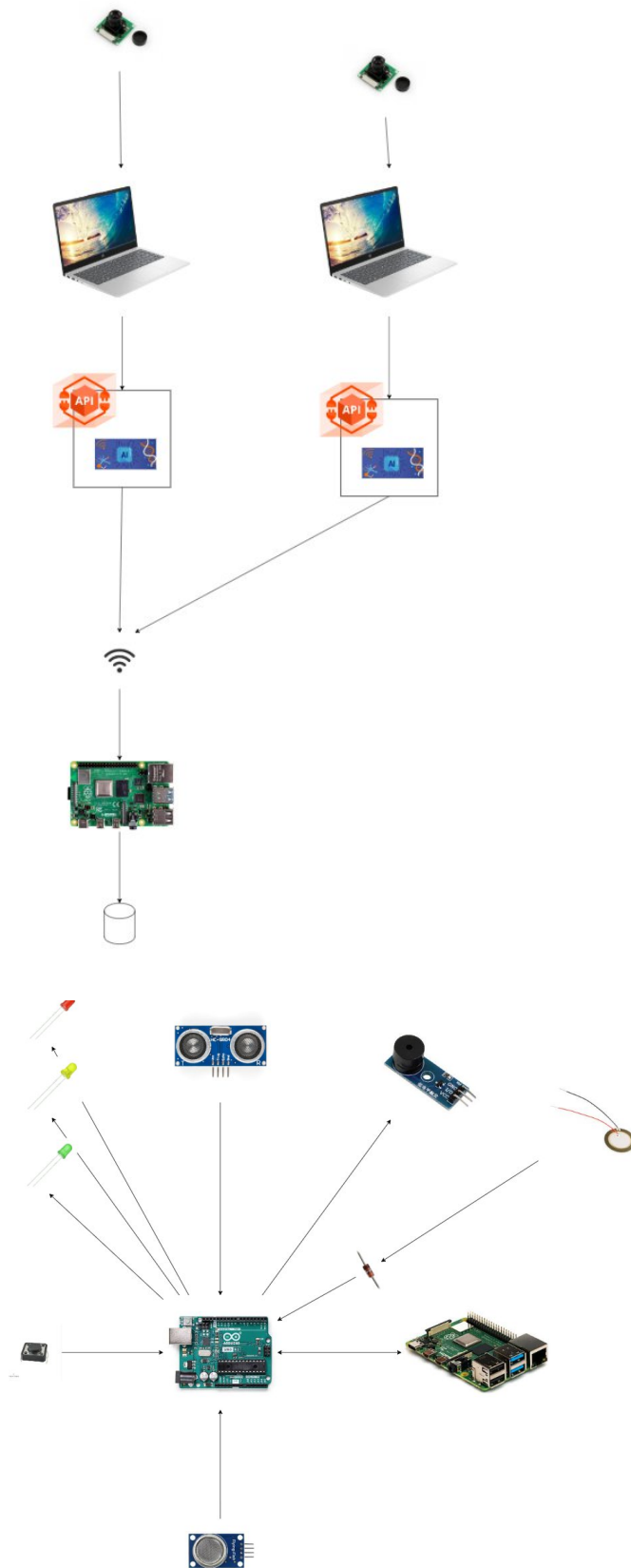
Puntos Extras:

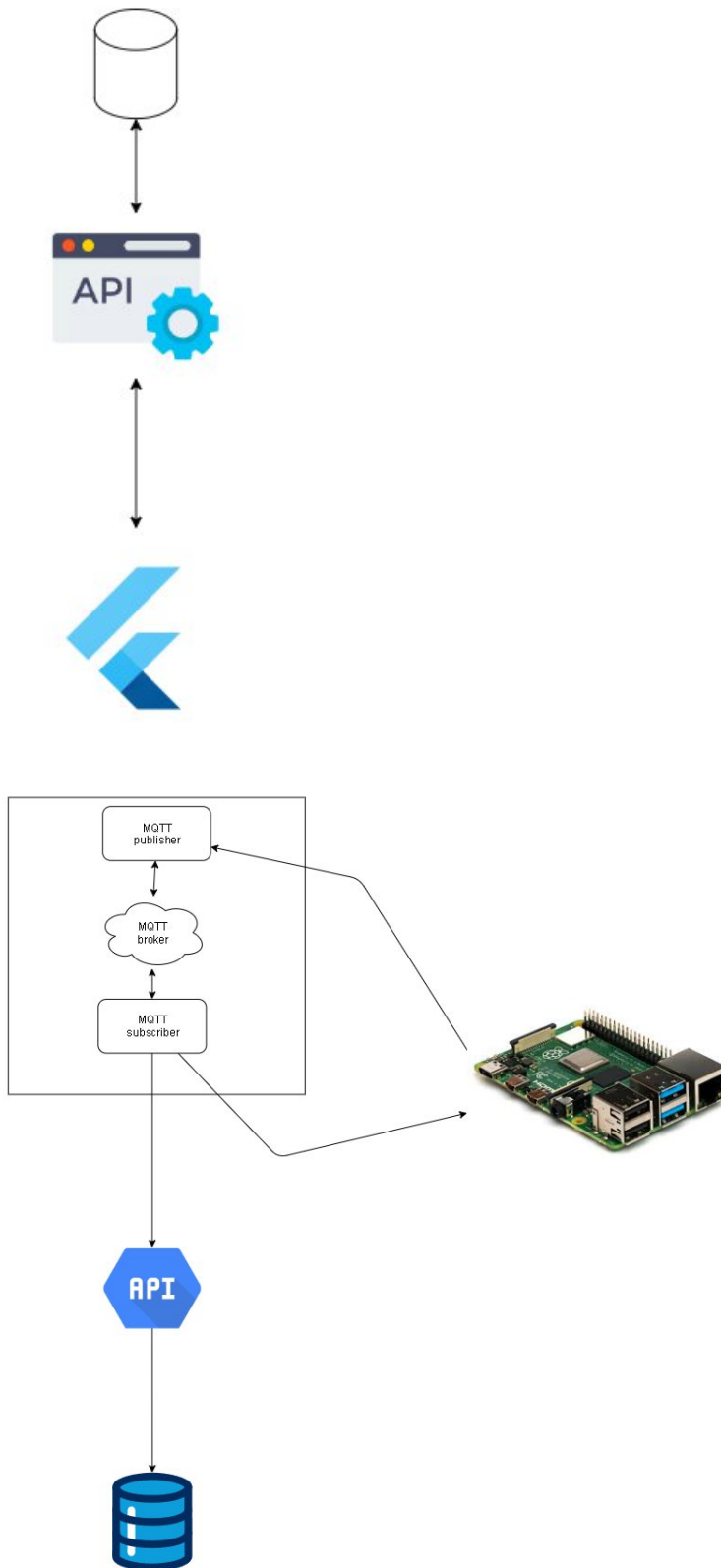
- Carrito de control remoto (puede ser comprado) en la maqueta con un excelente tracking en flutter (gps).
- Implementar una IA extra aparte de las ya implementadas (placas de autos, rostros, armas blancas), tiene que ser una IA innovadora.
- Distinción de buses no importando si van en la misma vía.

Para optar debe ser evaluado previamente, debe tener un buen proyecto.

Para más aclaraciones de los puntos extras consultar a los auxiliares.

Diagramas





IMPORTANTE:

Los demas funciones de la fase anterior deben de estar implementadas.

4.3 Entregables

Tipo	Descripción
Prototipo físico	Se realizará la documentación correspondiente a las modificaciones de la maqueta al agregar las cámaras.
Maqueta del prototipo propuesto	Importante recordar que el enfoque general es sobre un Sistema inteligente de Monitoreo de seguridad mediante IoT, por lo cual se necesita una maqueta física que encapsule los componentes utilizados en la fase y se oriente hacia el entorno del sistema.
Smart connected desing framework	Descripción de las capas de Smart Connected design Framework utilizadas en la fase.
Diagramas	Diagramas a criterios del estudiante que ayuden a evidenciar el flujo de la información.
Documentacion semanal de los aportes.	Se realiara la documentacion de lo que cada integrante aporte en el poyecto.

5. Metodología

1. Investigación, Selección y Diseño de Modelos de IA:

- **Benchmarking de Modelos:** Se investigarán y compararán los modelos de IA propuestos (YOLO, EasyOCR, MobileNetSSD, Haar Cascades, etc.) en función de su precisión, velocidad de inferencia y requisitos de hardware. Se realizarán pruebas preliminares en las laptops disponibles para identificar la mejor combinación para cada tarea (detección de placas, rostros, armas blancas).
- **Diseño del Flujo de Datos de IA:** Se definirá la arquitectura del software en las laptops. Esto incluye seleccionar el framework para la API local (Flask, FastAPI), diseñar el formato del payload JSON que se enviará a la Raspberry Pi (incluyendo

campos para `tipo_evento`, `metadatos`, `imagen_base64` o `url_imagen`) y establecer el protocolo de comunicación (HTTP POST a `192.168.0.5/api/info`).

- **Diseño de la Base de Datos Extendida:** Se ampliará el esquema de la base de datos existente para incluir las nuevas tablas (`detecciones_placas`, `detecciones_personas`, `detecciones_armas`). Se definirán las relaciones, los tipos de datos y los índices necesarios para optimizar las consultas analíticas.
- **Diseño de la Interfaz de Usuario (UI/UX) para Flutter:** Se crearán wireframes y mockups para las nuevas secciones de la app: las pantallas de alertas específicas para "Lista Negra" y "Arma Blanca", y los widgets del dashboard para los "Top de Personas" y "Top de Armas". Se definirá la lógica de actualización (push notifications o polling) para las alertas en tiempo real.

2. Desarrollo e Implementación de Componentes:

- **Desarrollo del Backend en Raspberry Pi:**
 - Se extenderá la API REST existente para incluir el nuevo endpoint `POST /api/info`, responsable de recibir, validar y almacenar los datos de las detecciones de IA.
 - Se implementará la lógica de negocio que, al recibir ciertos tipos de eventos (persona en lista negra, arma blanca), envíe un comando serial específico (ej: `"ALERTA_ROBO"`, `"ALERTA_ARMA"`) al Arduino UNO.
 - Se crearán nuevos endpoints para el frontend de Flutter, como `GET /api/top_personas` y `GET /api/top_armas`, que consulten la base de datos y devuelvan los datos agregados para los rankings.
- **Desarrollo del Módulo de IA en Laptops:**
 - Se configurarán los entornos de Python en las laptops y se instalarán las bibliotecas necesarias (OpenCV, TensorFlow/PyTorch, Ultralytics, etc.).
 - Se desarrollarán los scripts de captura de video (desde cámara USB o DroidCam) y de inferencia con los modelos seleccionados.
 - Se implementará el servicio ligero (API local) que empaquete los resultados de la detección y los envíe a la Raspberry Pi.
- **Desarrollo del Firmware en Arduino:**
 - Se actualizará el código del Arduino UNO para que escuche comandos seriales adicionales (`"ALERTA_ROBO"`, `"ALERTA_ARMA"`).
 - Se programarán patrones de sonido diferenciados en el zumbador para cada nuevo tipo de alerta, manteniendo los ya existentes.
- **Desarrollo del Frontend en Flutter:**
 - Se implementarán las nuevas pantallas y widgets en la aplicación móvil.
 - Se integrará la lógica para consumir los nuevos endpoints de la API y mostrar las alertas y rankings.
 - Se configurará el sistema de notificaciones push (o un mecanismo de polling eficiente) para alertar al usuario en tiempo real.

3. Documentación, Despliegue Final y Preparación para la Demostración:

- **Documentación Técnica:** Se actualizará toda la documentación del proyecto, incluyendo diagramas de arquitectura actualizados, especificaciones de la API, esquemas de la base de datos y manuales de usuario para la app Flutter.

- **Despliegue y Configuración Final:** Se realizará la configuración final de todos los componentes en la maqueta. Se asegurará que todas las laptops, la Raspberry Pi, el Arduino y la app Flutter estén correctamente conectados y sincronizados.
- **Preparación de la Demostración:** Se preparará un guion para la demostración final, que muestre de forma clara y concisa el funcionamiento de todas las nuevas funcionalidades: la detección de un objeto por IA, la alerta física en el zumbador, la notificación en la app, la visualización en el dashboard de Flutter y la gráfica en Grafana. Se prepararán escenarios de prueba controlados para garantizar una presentación exitosa.

6. Desarrollo de Habilidades Blandas

Para complementar el desarrollo técnico, esta sección se centra en las habilidades blandas que los estudiantes deberán mejorar a lo largo del proyecto, como la comunicación, el liderazgo, y la colaboración en equipo.

6.1 Proyectos en Grupo

Este proyecto se desarrollará en equipos, fomentando la colaboración y distribución de responsabilidades bajo un modelo flexible, adaptado a las necesidades del curso.

Estructura sugerida:

- **Roles no fijos:** Los integrantes rotarán funciones (coordinación, desarrollo, documentación) para garantizar participación equitativa.
- **Autogestión:** Cada equipo definirá su método de trabajo (reuniones semanales, división de tareas).

6.1.1 Herramientas de Organización

- **GitHub:** Control de versiones para código y documentación.
- **Plataformas colaborativas:** Google Drive o Trello para asignar tareas y compartir avances (opcional).

6.1.2 Comunicación y Retroalimentación

- **Revisiones periódicas:** Presentaciones breves en clase para mostrar progresos y recibir feedback del profesor.

- **Resolución de conflictos:** Los equipos deberán resolver discrepancias internamente, con apoyo del docente si es necesario.

7. Cronograma

FASE 1

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	sábado 20 de Septiembre	sábado 20 de Septiembre
Elaboración	sábado 20 de Septiembre	viernes 17 de Octubre
Calificación	sábado 18 de Octubre	sábado 18 de Octubre

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Tema	Descripción	Cumple (Si/No)
Cumplimiento de la tecnología establecida	Uso de arduino, raspberri, flutter y componentes eléctricos.	
Entrega de la fase	Entregar la Fase 2.	
Prototipo Fisico	Se debe entregar el protipo fisico en funcionamiento.	
Documentacion obligatoria	Se debe entregar en la documentación de las modificaciones de la maqueta, los semanales, los diagramas que crea correspondiente.	
Entregables	Se deberán de enviar todos los entregables de la documentación.	
Github	Todo el código utilizado y la documentación deberá ser subido a un repositorio de github.	
Nombre del repositorio	ARQUI2B_2S2025_G<INICIAL><NUMERO>	

Usuarios de Github	Agregar el usuario del auxiliar como colaborador a su repositorio de github	
Nombre Carpeta de la fase	Debido a que se usará el mismo repositorio durante todo el semestre, se solicita que contenga 3 carpetas, en las cuales se presenta en cada fase.	

- Ejemplo:
 - Inicial A o L
 - Grupo 1-4
 - ARQUI2B_2S2025_GA1
- Github
 - Janjo25
 - AlexCB-16
- Nombre de la Carpeta
 - FASE 1
 - FASE 2
 - FASE 3
- Calificación será grupal y otra parte individual

8.2 Valores

En el desarrollo del proyecto, se espera que cada estudiante demuestre honestidad académica y profesionalismo. Por lo tanto, se establecen los siguientes principios:

1. **Originalidad del Trabajo**
 - Cada estudiante o equipo debe desarrollar su propio código y/o documentación, aplicando los conocimientos adquiridos en el curso.
2. **Prohibición de Copias y Plagio**
 - Si se detecta la copia total o parcial del código, documentación o cualquier otro entregable, la calificación será de **0 puntos**.
 - Esto incluye la reproducción de código entre compañeros, la reutilización de proyectos de semestres anteriores o el uso de código externo sin la debida referencia.
3. **Uso Responsable de Recursos Externos**
 - El uso de bibliotecas, frameworks y ejemplos de código externos está permitido, siempre y cuando se referencian correctamente y se comprendan plenamente. (Consultar con el catedrático su política)
4. **Revisión y Detección de Plagio**
 - Se podrán utilizar herramientas automatizadas y revisiones manuales para identificar similitudes en los proyectos.
 - En caso de sospecha, el estudiante deberá justificar su código y demostrar su desarrollo individual o en equipo. Si este extremo no es comprobable la calificación será de **0 puntos**.

Al detectarse estos aspectos se informará al catedrático del curso quien realizará las acciones que considere oportunas.

8.3 Comentarios Generales

- Todas las aclaraciones se realizarán en clase, por lo que deberán acatar todas las instrucciones escritas y verbales al momento de explicar el proyecto.
- Se calificará solamente lo que sea completamente funcional.
- Se calificará los commits individualmente.